

**РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
ФАКУЛЬТЕТ ФИЗИКО-МАТЕМАТИЧЕСКИХ И
ЕСТЕСТВЕННЫХ НАУК**

Лабораторная работа №.10

Работа с файлами средствами Nasm.

Левищева Анастасия Петровна, НКАбд-02-25

Содержание

Список иллюстраций.....	3
1. Цель работы.....	4
2. Задание.....	5
3. Теоретическое введение.....	6
4. Выполнение лабораторной работы.....	13
5. Выводы.....	17
Список литературы.....	18

Список иллюстраций

Рис.1.Каталог.....	13
Рис.2.Исполняемый файл(1).....	13
Рис.3.Запрет на выполнение.....	13
Рис.4.Разрешение выполнения.....	14
Рис.5.Изменение прав доступа.....	14
Рис.6.Файл sw.asm.....	15
Рис.7.Текст программы.....	15
Рис.8. Исполняемый файл(2).....	16
Рис.9.Проверка.....	16

1. Цель работы

Приобретение навыков написания программ для работы с файлами.

2. Задание

1. Напишите программу, работающую по следующему алгоритму:

- Вывод приглашения “Как Вас зовут?”
- ввести с клавиатуры свои фамилию и имя
- создать файл с именем name.txt
- записать в файл сообщение “Меня зовут”
- дописать в файл строку введенную с клавиатуры
- закрыть файл

Создать исполняемый файл и проверить его работу. Проверить наличие файла и его содержимое с помощью команд `ls` и `cat`.

3. Теоретическое введение

3.1. Права доступа к файлам

ОС GNU/Linux является многопользовательской операционной системой. И для обеспечения защиты данных одного пользователя от действий других пользователей существуют специальные механизмы разграничения доступа к файлам. Кроме ограничения доступа, данный механизм позволяет разрешить другим пользователям доступ данным для совместной работы.

Права доступа определяют набор действий (чтение, запись, выполнение), разрешённых для выполнения пользователям системы над файлами. Для каждого файла пользователь может входить в одну из трех групп: владелец, член группы владельца, все остальные. Для каждой из этих групп может быть установлен свой набор прав доступа. Владелцем файла является его создатель. Для предоставления прав доступа другому пользователю или другой группе командой

```
chown [ключи] <новый_пользователь>[:новая_группа] <файл>
```

или

```
chgrp [ключи] < новая_группа > <файл>
```

Набор прав доступа задается тройками битов и состоит из прав на чтение, запись и исполнение файла. В символьном представлении он имеет вид строк `gwx`, где вместо любого символа может стоять дефис. Всего возможно 8 комбинаций, приведенных в таблице 10.1. Буква означает наличие права (установлен в единицу второй бит триады `r` — чтение, первый бит `w` — запись, нулевой бит `x` — исполнение), а дефис означает отсутствие права (нулевое значение соответствующего бита). Также права доступа могут быть представлены как восьмеричное число. Так, права доступа `rw`- (чтение и запись, без исполнения) понимаются как три двоичные цифры `110` или как восьмеричная цифра `6`.

Таблица 10.1. Двоичный, буквенный и восьмеричный способ записи триады прав доступа

Двоичный	Буквенный	Восьмеричный
111	гwx	7
110	гw-	6
101	г-x	5
100	г--	4
011	-wx	3
010	-w-	2
001	--x	1
000	---	0

Полная строка прав доступа в символьном представлении имеет вид:

<права_владельца> <права_группы> <права_остальных>

Так, например, права гwx г-x --x выглядят как двоичное число 111 101 001, или восьмеричное 751. Свойства (атрибуты) файлов и каталогов можно вывести на терминал с помощью команды `ls` с ключом `-l`. Так например, чтобы узнать права доступа к файлу `README` можно узнать с помощью следующей команды:

```
$ls -l /home/debugger/README
-rwxr-xr-- 1 debugger users 0 Feb 14 19:08 /home/debugger/README
```

В первой колонке показаны текущие права доступа, далее указан владелец файла и группа:

-	гwx	г-x	г-x	l debugger user
тип	владелец	группа	остальные	
type	owner	group	others	

Тип файла определяется первой позицией, это может быть: каталог — `d`, обычный файл — дефис (`-`) или символьная ссылка на другой файл — `l`. Следующие 3 набора по 3 символа определяют конкретные права для конкретных групп: `г` — разрешено чтение файла, `w` — разрешена запись в файл; `x` — разрешено исполнение файл и дефис (`-`) — право не дано. Для изменения прав доступа служит команда `chmod`, которая понимает как символьное, так и числовое указание прав. Для того чтобы назначить файлу `/home/debugger/README` права `rw-r`, то есть разрешить владельцу чтение и запись, группе только чтение, остальным пользователям — ничего:

```
$chmod 640 README # 110 100 000 == 640 == rw-r-----
$ls -l README
-rw-r--r 1 debugger users 0 Feb 14 19:08 /home/debugger/README
```

В символьном представлении есть возможность явно указывать какой группе какие права необходимо добавить, отнять или присвоить. Например, чтобы добавить право на исполнение файла README группе и всем остальным:

```
$chmod go+x README
$ls -l README
-rw-r-x--x 1 debugger users 0 Feb 14 19:08 /home/debugger/README
```

Формат символьного режима:

```
chmod <категория><действие><набор_прав><файл>
```

Возможные значения аргументов команды представлены в таблице 10.2.

Таблица 10.2. Возможные значения аргументов команды chmod

Категория	Обозначение	Значение
Принадлежность	u	Владелец
	g	Группа владельца
	o	Прочие пользователи
	a	Все пользователи, то есть «a» эквивалентно «ugo»
Действие	+	Добавить набор прав
	-	Отменить набор прав
	=	Назначить набор прав
Право	r	Право на чтение
	w	Право на запись
	x	Право на исполнение

3.2. Работа с файлами средствами Nasm

В операционной системе Linux существуют различные методы управления файлами, например, такие как создание и открытие файла, только для чтения или для чтения и записи, добавления в существующий файл, закрытия и удаления файла, предоставление прав доступа.

Обработка файлов в операционной системе Linux осуществляется за счет использования определенных системных вызовов. Для корректной работы и

доступа к файлу при его открытии или создании, файлу присваивается уникальный номер (16-битное целое число) – дескриптор файла. В таблице 10.3 приведены системные вызовы для обработки файлов.

Таблица 10.3. Системные вызовы для обработки файлов

Имя системного вызова	eax	ebx	ecx	edx
sys_read	3	дескриптор файла	адрес в памяти	количество байтов
sys_write	4	дескриптор файла	строка	количество байтов
sys_open	5	имя файла	режим доступа к файлу	права доступа к файлу
sys_close	6	дескриптор файла	—	—
sys_creat	8	имя файла	права доступа к файлу	—
sys_unlink	10	имя файла	—	—
sys_lseek	19	имя файла	значение смещения в байтах	позиция для смещения

Общий алгоритм работы с системными вызовами в Nasm можно представить в следующем виде:

1. Поместить номер системного вызова в регистр EAX;
2. Поместить аргументы системного вызова в регистрах EBX, ECX и EDX;
3. Вызов прерывания (int 80h);
4. Результат обычно возвращается в регистр EAX.

3.2.1. Открытие и создание файла

Для создания и открытия файла служит системный вызов sys_creat, который использует следующие аргументы: права доступа к файлу в регистре ECX, имя файла в EBX и номер системного вызова sys_creat (8) в EAX.

```

mov ecx, 0777o    ; установка прав доступа
mov ebx, filename ; имя создаваемого файла
mov eax, 8        ; номер системного вызова `sys_creat`
int 80h           ; вызов ядра

```

Для открытия существующего файла служит системный вызов `sys_open`, который использует следующие аргументы: права доступа к файлу в регистре `EDX`, режим доступа к файлу в регистр `ECX`, имя файла в `EBX` и номер системного вызова `sys_open` (5) в `EAX`.

Среди режимов доступа к файлам чаще всего используются:

- (0) – `O_RDONLY` (открыть файл в режиме только для чтения);
- (1) – `O_WRONLY` – (открыть файл в режиме только записи);
- (2) – `O_RDWR` – (открыть файл в режиме чтения и записи).

Системный вызов возвращает файловый дескриптор открытого файла в регистр `EAX`. В случае ошибки, код ошибки также будет находиться в регистре `EAX`.

```
mov ecx, 0      ; режим доступа (0 - только чтение)
mov ebx, filename ; имя открываемого файла
mov eax, 5      ; номер системного вызова 'sys_open'
int 80h         ; вызов ядра
```

3.2.2. Запись в файл

Для записи в файл служит системный вызов `sys_write`, который использует следующие аргументы: количество байтов для записи в регистре `EDX`, строку содержимого для записи `ECX`, файловый дескриптор в `EBX` и номер системного вызова `sys_write` (4) в `EAX`.

Системный вызов возвращает фактическое количество записанных байтов в регистр `EAX`. В случае ошибки, код ошибки также будет находиться в регистре `EAX`.

Прежде чем записывать в файл, его необходимо создать или открыть, что позволит получить дескриптор файла.

```
mov ecx, 0777o    ; Создание файла.
mov ebx, filename ; в случае успешного создания файла,
mov eax, 8        ; в регистр eax запишется дескриптор файла
int 80h

mov edx, 12        ; количество байтов для записи
mov ecx, msg       ; адрес строки для записи в файл
mov ebx, eax       ; дескриптор файла
mov eax, 4        ; номер системного вызова 'sys_write'
int 80h           ; вызов ядра
```

3.2.3. Чтение файла

Для чтения данных из файла служит системный вызов `sys_read`, который использует следующие аргументы: количество байтов для чтения в регистре `EDX`, адрес в памяти для записи прочитанных данных в `ECX`, файловый дескриптор в `EBX` и номер системного вызова `sys_read` (3) в `EAX`. Как и для записи, прежде чем читать из файла, его необходимо открыть, что позволит получить дескриптор файла.

```
mov  ecx, 0          ; Открытие файла.
mov  ebx, filename    ; в случае успешного открытия файла,
mov  eax, 5          ; в регистр EAX запишется дескриптор файла
int  80h

mov  edx, 12         ; количество байтов для чтения
mov  ecx, fileCont    ; адрес в памяти для записи прочитанных данных
mov  ebx, eax         ; дескриптор файла
mov  eax, 3          ; номер системного вызова `sys_read`
int  80h             ; вызов ядра
```

3.2.4. Закрытие файла

Для правильного закрытия файла служит системный вызов `sys_close`, который использует один аргумент – дескриптор файла в регистре `EBX`. После вызова ядра происходит удаление дескриптора файла, а в случае ошибки, системный вызов возвращает код ошибки в регистр `EAX`.

```
mov  ecx, 0          ; Открытие файла.
mov  ebx, filename    ; в случае успешного открытия файла,
mov  eax, 5          ; в регистр EAX запишется дескриптор файла
int  80h

mov  ebx, eax         ; дескриптор файла
mov  eax, 6          ; номер системного вызова `sys_close`
int  80h             ; вызов ядра
```

3.2.5. Изменение содержимого файла

Для изменения содержимого файла служит системный вызов `sys_lseek`, который использует следующие аргументы: исходная позиция для смещения

EDX, значение смещения в байтах в ECX, файловый дескриптор в EBX и номер системного вызова `sys_lseek` (19) в EAX.

Значение смещения можно задавать в байтах. Значения, обозначающие исходную позиции могут быть следующими:

- (0) – `SEEK_SET` (начало файла);
- (1) – `SEEK_CUR` (текущая позиция);
- (2) – `SEEK_END` (конец файла).

В случае ошибки, системный вызов возвращает код ошибки в регистр EAX.

```
mov ecx, 1          ; Открытие файла (1 - для записи).
mov ebx, filename
mov eax, 5
int 80h

mov edx, 2          ; значение смещения -- конец файла
mov ecx, 0          ; смещение на 0 байт
mov ebx, eax        ; дескриптор файла
mov eax, 19         ; номер системного вызова `sys_lseek`
int 80h            ; вызов ядра

mov edx, 9          ; Запись в конец файла
mov ecx, msg        ; строки из переменной `msg`
mov eax, 4
int 80h
```

3.2.6. Удаление файла

Удаление файла осуществляется системным вызовом `sys_unlink`, который использует один аргумент – имя файла в регистре EBX.

```
mov ebx, filename  ; имя файла
mov eax, 10        ; номер системного вызова `sys_unlink`
int 80h           ; вызов ядра
```

4. Выполнение лабораторной работы

1. Создадим каталог для программ лабораторной работы № 10, перейдем в него и создадим файлы lab10-1.asm, readme-1.txt и readme-2.txt:

```
aplevishcheva@debian:~/work/arch-pc/lab10$ touch lab10-1.asm readme-1.txt readme-2.txt
aplevishcheva@debian:~/work/arch-pc/lab10$ ls
lab10-1.asm  readme-1.txt  readme-2.txt
aplevishcheva@debian:~/work/arch-pc/lab10$ █
```

Рис.1.Каталог

2. Введем в файл lab10-1.asm текст программы из листинга 10.1 (Программа записи в файл сообщения). Создадим исполняемый файл и проверим его работу:

```
aplevishcheva@debian:~/work/arch-pc/lab10$ nasm -f elf lab10-1.asm
aplevishcheva@debian:~/work/arch-pc/lab10$ ld -m elf_i386 -o lab10-1 lab10-1.o
aplevishcheva@debian:~/work/arch-pc/lab10$ ./lab10-1
Введит строку для записи в файл: 89
aplevishcheva@debian:~/work/arch-pc/lab10$ ls -l
итого 28
-rw-rw-r-- 1 aplevishcheva aplevishcheva 3942 ноя  8 17:10 in_out.asm
-rwxrwxr-x 1 aplevishcheva aplevishcheva 9164 дек 14 16:11 lab10-1
-rw-rw-r-- 1 aplevishcheva aplevishcheva 552 дек 14 16:11 lab10-1.asm
-rw-rw-r-- 1 aplevishcheva aplevishcheva 1472 дек 14 16:11 lab10-1.o
-rw-rw-r-- 1 aplevishcheva aplevishcheva 3 дек 14 16:11 readme-1.txt
-rw-rw-r-- 1 aplevishcheva aplevishcheva 0 дек 14 15:57 readme-2.txt
aplevishcheva@debian:~/work/arch-pc/lab10$ cat readme-1.txt
89
aplevishcheva@debian:~/work/arch-pc/lab10$ █
```

Рис.2.Исполняемый файл(1)

Программа работает корректно.

3. С помощью команды chmod изменим права доступа к исполняемому файлу lab10-1, запретив его выполнение и попытаемся выполнить файл:

```
aplevishcheva@debian:~/work/arch-pc/lab10$ chmod 666 *lab10-1
aplevishcheva@debian:~/work/arch-pc/lab10$ ls -l lab10-1
-rw-rw-rw- 1 aplevishcheva aplevishcheva 9164 дек 14 16:11 lab10-1
aplevishcheva@debian:~/work/arch-pc/lab10$ ./lab10-1
bash: ./lab10-1: Отказано в доступе
aplevishcheva@debian:~/work/arch-pc/lab10$ █
```

Рис.3.Запрет на выполнение

Всем пользователям было запрещено выполнение файла.

4. С помощью команды chmod изменим права доступа к файлу lab10-1.asm с исходным текстом программы, добавив права на исполнение и попытаемся выполнить его:

```

aplevishcheva@debian:~/work/arch-pc/lab10$ chmod 777 *lab10-1
aplevishcheva@debian:~/work/arch-pc/lab10$ ls -l lab10-1
-rwxrwxrwx 1 aplevishcheva aplevishcheva 9164 дек 14 16:11 lab10-1
aplevishcheva@debian:~/work/arch-pc/lab10$ ./lab10-1
Введит строку для записи в файл: 89
aplevishcheva@debian:~/work/arch-pc/lab10$ ls -l
итого 28
-rw-rw-r-- 1 aplevishcheva aplevishcheva 3942 ноя  8 17:10 in_out.asm
-rwxrwxrwx 1 aplevishcheva aplevishcheva 9164 дек 14 16:11 lab10-1
-rw-rw-r-- 1 aplevishcheva aplevishcheva  552 дек 14 16:11 lab10-1.asm
-rw-rw-r-- 1 aplevishcheva aplevishcheva 1472 дек 14 16:11 lab10-1.o
-rw-rw-r-- 1 aplevishcheva aplevishcheva    3 дек 14 16:30 readme-1.txt
-rw-rw-r-- 1 aplevishcheva aplevishcheva    0 дек 14 15:57 readme-2.txt
aplevishcheva@debian:~/work/arch-pc/lab10$ cat readme-1.txt
89
aplevishcheva@debian:~/work/arch-pc/lab10$ █

```

Рис.4.Разрешение выполнения

Всем пользователям было разрешено чтение, запись и выполнение. Программа была успешно выполнена.

5. В соответствии с вариантом 4 в таблице 10.4 предоставим права доступа к файлу readme-1.txt представленные в символьном виде, а для файла readme-2.txt – в двоичном виде и проверим правильность выполнения с помощью команды ls -l:

```

aplevishcheva@debian:~/work/arch-pc/lab10$ chmod a+rwx readme-1.txt
aplevishcheva@debian:~/work/arch-pc/lab10$ ls -l
итого 28
-rw-rw-r-- 1 aplevishcheva aplevishcheva 3942 ноя  8 17:10 in_out.asm
-rwxrwxrwx 1 aplevishcheva aplevishcheva 9164 дек 14 16:11 lab10-1
-rw-rw-r-- 1 aplevishcheva aplevishcheva  552 дек 14 16:11 lab10-1.asm
-rw-rw-r-- 1 aplevishcheva aplevishcheva 1472 дек 14 16:11 lab10-1.o
-rwxrwxrwx 1 aplevishcheva aplevishcheva    3 дек 14 16:30 readme-1.txt
-rw-rw-r-- 1 aplevishcheva aplevishcheva    0 дек 14 15:57 readme-2.txt
aplevishcheva@debian:~/work/arch-pc/lab10$ chmod 777 readme-2.txt
aplevishcheva@debian:~/work/arch-pc/lab10$ ls -l
итого 28
-rw-rw-r-- 1 aplevishcheva aplevishcheva 3942 ноя  8 17:10 in_out.asm
-rwxrwxrwx 1 aplevishcheva aplevishcheva 9164 дек 14 16:11 lab10-1
-rw-rw-r-- 1 aplevishcheva aplevishcheva  552 дек 14 16:11 lab10-1.asm
-rw-rw-r-- 1 aplevishcheva aplevishcheva 1472 дек 14 16:11 lab10-1.o
-rwxrwxrwx 1 aplevishcheva aplevishcheva    3 дек 14 16:30 readme-1.txt
-rwxrwxrwx 1 aplevishcheva aplevishcheva    0 дек 14 15:57 readme-2.txt
aplevishcheva@debian:~/work/arch-pc/lab10$ █

```

Рис.5.Изменение прав доступа

Все выполнено правильно.

Задания для самостоятельной работы.

Задание 1(стр.5)

Создадим файл sw.asm:

```
aplevishcheva@debian:~/work/arch-pc/lab10$ touch sw.asm
aplevishcheva@debian:~/work/arch-pc/lab10$ ls
in_out.asm  lab10-1  lab10-1.asm  lab10-1.o  readme-1.txt  readme-2.txt  sw.asm
aplevishcheva@debian:~/work/arch-pc/lab10$ █
```

Рис.6.Файл sw.asm

Напишем в нем текст программы с требуемым алгоритмом:

```
%include 'in_out.asm'

section .data

    filename db 'name.txt', 0
    msg      db 'Как Вас зовут? ', 0

section .bss

contents resb 255

section .text
global _start

_start:

    mov eax, msg
    call sprint

    mov ecx, contents
    mov edx, 255
    call sread

    mov ecx, 2
    mov ebx, filename
    mov eax, 5
    int 80h

    mov esi, eax

    mov edx, 13
    mov ecx, contents
    mov ebx, esi
    mov eax, 4
    int 80h

    mov ebx, esi
    mov eax, 6
    int 80h

    call quit
```

Рис.7.Текст программы

Создадим исполняемый файл и проверим его работу:

```
aplevishcheva@debian:~/work/arch-pc/lab10$ nasm -f elf sw.asm
aplevishcheva@debian:~/work/arch-pc/lab10$ ld -m elf_i386 -o sw sw.o
aplevishcheva@debian:~/work/arch-pc/lab10$ ./sw
Как Вас зовут? Анастасия
```

Рис.8. Исполняемый файл(2)

Проверим наличие файла и его содержимое с помощью команд `ls` и `cat`:

```
aplevishcheva@debian:~/work/arch-pc/lab10$ ls -l
итого 52
-rw-rw-r-- 1 aplevishcheva aplevishcheva 3942 ноя  8 17:10 in_out.asm
-rwxrwxrwx 1 aplevishcheva aplevishcheva 9164 дек 14 16:11 lab10-1
-rw-rw-r-- 1 aplevishcheva aplevishcheva  552 дек 14 16:11 lab10-1.asm
-rw-rw-r-- 1 aplevishcheva aplevishcheva 1472 дек 14 16:11 lab10-1.o
-rw-rw-r-- 1 aplevishcheva aplevishcheva   13 дек 14 17:47 name.txt
-rwxrwxrwx 1 aplevishcheva aplevishcheva    3 дек 14 16:30 readme-1.txt
-rwxrwxrwx 1 aplevishcheva aplevishcheva    0 дек 14 15:57 readme-2.txt
-rwxrwxr-x 1 aplevishcheva aplevishcheva 9120 дек 14 17:41 sw
-rw-rw-r-- 1 aplevishcheva aplevishcheva  478 дек 14 17:41 sw.asm
-rw-rw-r-- 1 aplevishcheva aplevishcheva 1408 дек 14 17:41 sw.o
aplevishcheva@debian:~/work/arch-pc/lab10$ cat name.txt
Anastasiya
aplevishcheva@debian:~/work/arch-pc/lab10$ █
```

Рис.9.Проверка

Программа работает корректно.

5. Выводы

В ходе выполнения лабораторной работы была успешно освоена работа с файлами средствами Nasm. Также были получены базовые знания о правах доступа к файлам.

Список литературы

1. GDB: The GNU Project Debugger. — URL: <https://www.gnu.org/software/gdb/>.
2. GNU Bash Manual. — 2016. — URL: <https://www.gnu.org/software/bash/manual/>.
3. Midnight Commander Development Center. — 2021. — URL: <https://midnight-commander.org/>.
4. NASM Assembly Language Tutorials. — 2021. — URL: <https://asmtutor.com/>.
5. Newham C. Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 с. — (In a Nutshell). — ISBN 0596009658. — URL: <http://www.amazon.com/Learningbash-Shell-Programming-Nutshell/dp/0596009658>.
6. Robbins A. Bash Pocket Reference. — O'Reilly Media, 2016. — 156 с. — ISBN 978-1491941591.
7. The NASM documentation. — 2021. — URL: <https://www.nasm.us/docs.php>.
8. Zarrelli G. Mastering Bash. — Packt Publishing, 2017. — 502 с. — ISBN 9781784396879.
9. Колдаев В. Д., Лупин С. А. Архитектура ЭВМ. — М. : Форум, 2018.
10. Куляс О. Л., Никитин К. А. Курс программирования на ASSEMBLER. — М. : Солон-Пресс, 2017.
11. Новожилов О. П. Архитектура ЭВМ и систем. — М. : Юрайт, 2016.
12. Расширенный ассемблер: NASM. — 2021. — URL: <https://www.opennet.ru/docs/RUS/nasm/>.
13. Робачевский А., Немнюгин С., Стесик О. Операционная система UNIX. — 2-е изд. — БХВПетербург, 2010. — 656 с. — ISBN 978-5-94157-538-1.
14. Столяров А. Программирование на языке ассемблера NASM для ОС Unix. — 2-е изд. — М. : МАКС Пресс, 2011. — URL: http://www.stolyarov.info/books/asm_unix.
15. Таненбаум Э. Архитектура компьютера. — 6-е изд. — СПб. : Питер, 2013. — 874 с. — (Классика Computer Science).
16. Таненбаум Э., Бос Х. Современные операционные системы. — 4-е изд. — СПб. : Питер, 2015. — 1120 с. — (Классика Computer Science).